

实验二

实验目的

本章实验的主要目的是掌握图像分割任务难点，了解如何使用深度学习解决相关问题。掌握不同图像分割神经网络架构的设计原理与核心思想，熟悉使用MindSpore深度学习框架实现深度学习实验的一般流程。

实验内容

任务一：按照华为平台实验手册进行操作

要求：熟悉实验环境，掌握Unet图像分割原理及程序流程

具体：记录并观察记录实验结果，如损失函数随迭代轮数变化等

任务二：Unet-5模型性能分析

要求：调节实验参数，进行试验对比及分析

实验参数包括：训练数据比例，批次大小，迭代轮数，学习速率

任务三：完成华为平台实验手册思考题

要求：尝试自动动手编写代码实现

具体：理解原理，记录实验结果并分析

实验过程

任务一：

原理介绍：

OTSU：

大津阈值法（OTSU），又称作最大类间方差法，是一种图像二值化分割阈值的算法，由日本学者大津于1979年提出。大津阈值法是具有统计意义上的最佳分割阈值。其核心思想就是使类间方差最大，按照大津阈值法求得的阈值进行图像二值分割以区分前后背景，前景与背景图像的类间方差最大。该算法要求被分割的物体颜色纹理比较紧凑，类内方差小，对于一些文本图像的处理（比如车牌、指纹）效果很好。

Unet网络结构：

Unet是一种改进的全卷积网络（FCN）用于图像分割任务。由Olaf Ronneberger等人在论文《U-Net: Convolutional Networks for Biomedical Image Segmentation》中提出。U-Net的网络结构和论文可以参考其官网：<https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>；U-Net得名于它的网络结构图，类似一个英文字母“U”。左边是一个从上到下，一步一步从原始图像抽取特征（即原始图像本质信息）的过程；右边是一个从下到上，一步一步从图像特征还原目标信息的过程。U-Net网络主要分为encoder和decoder两个部分。网络结构如下所示：

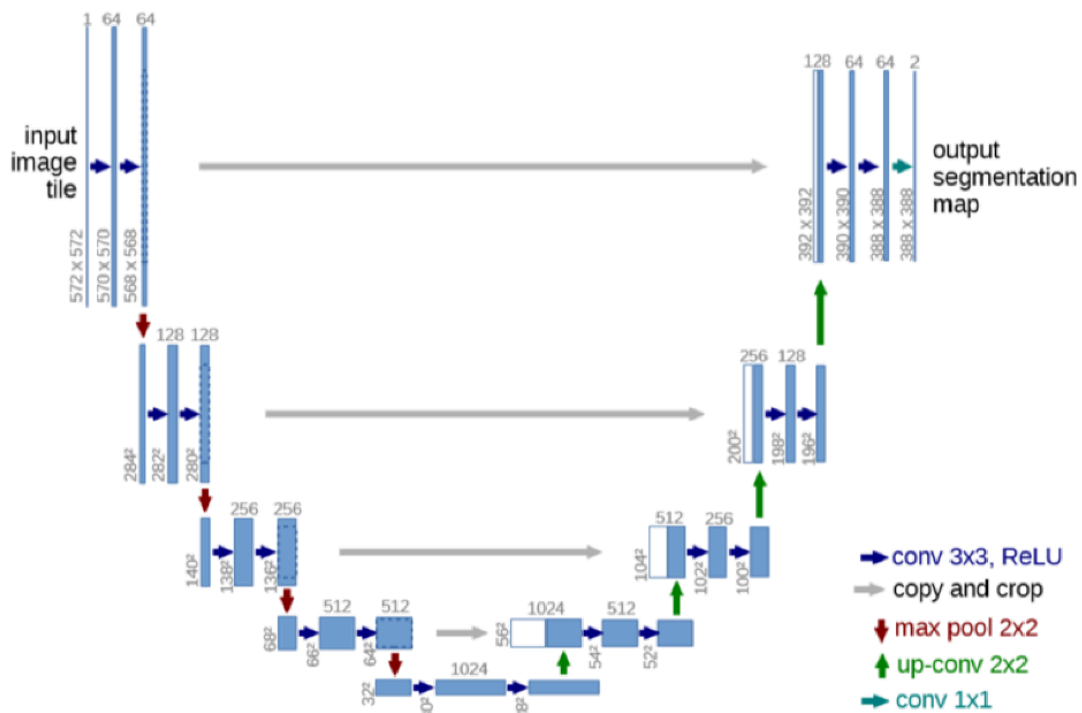
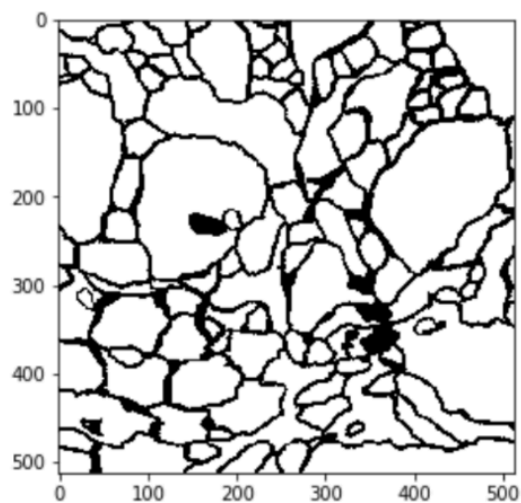
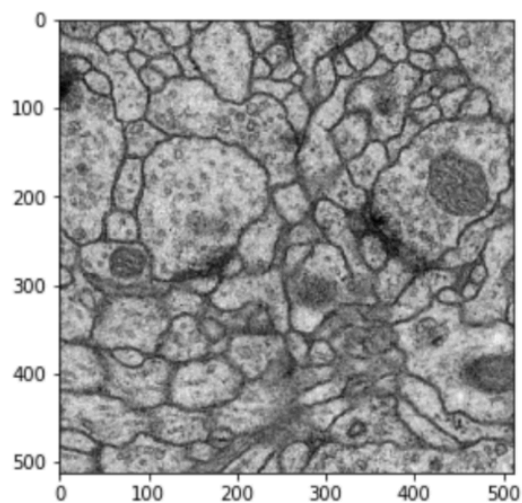


图2-3 Unet 网络结构

①打印图像和标签的形状，展示第一张图像和标签：

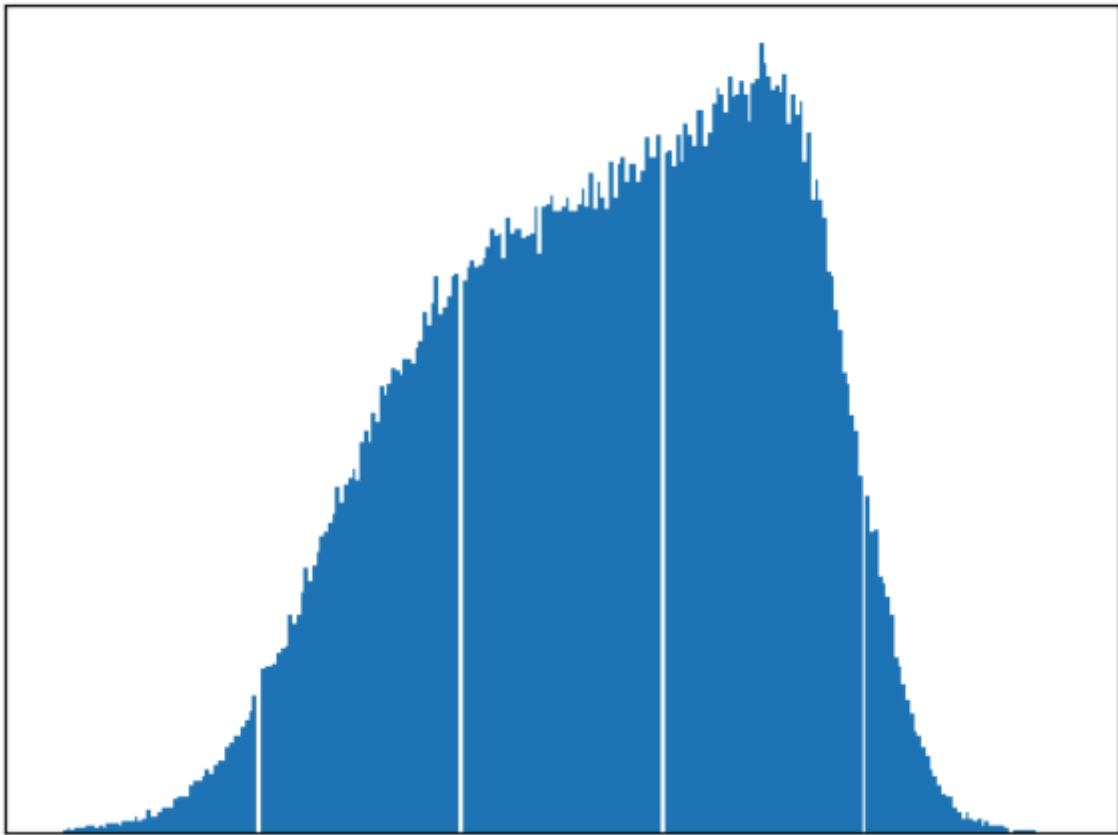
```
(30, 512, 512)
(30, 512, 512)
```



可以看到图像的height和weight都是512，这张图像的label是30

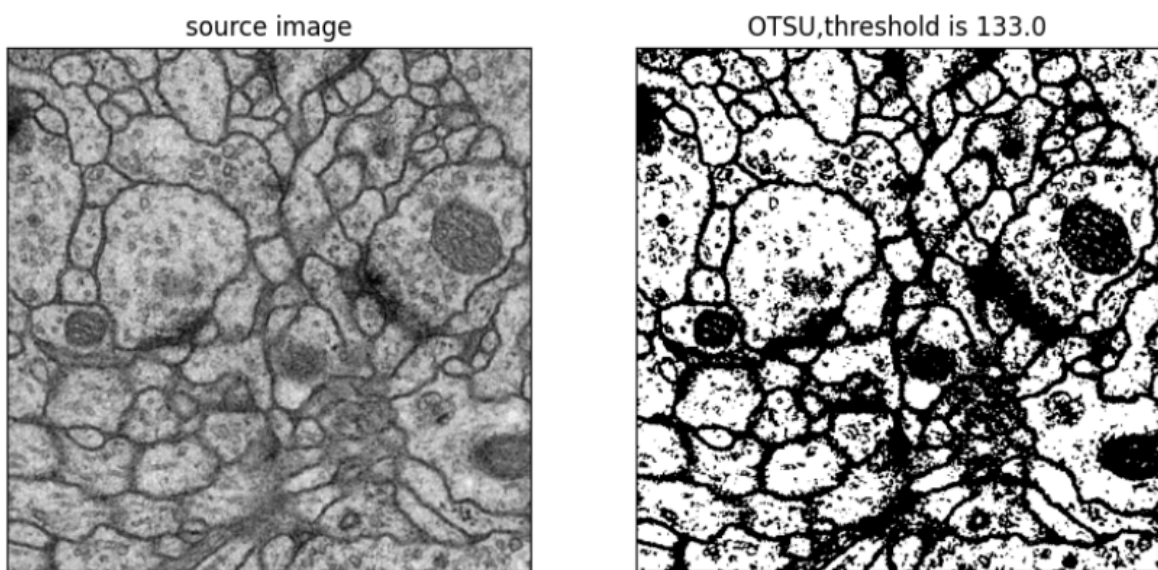
②绘制一张训练集中的直方图：

Histogram



发现该数据集的图像的直方图很连续，并没有出现明显的波谷，使用传统基于统计的阈值分割算法可能并不能取得好的效果。

③基于OTSU的图像二值化分割：



可以看到分割的结果并不算理想

④使用基于神经网络的图像分割算法。首先进行超参数配置和、各个模块搭建、定义损失函数等等

⑤数据项预处理

⑥模型训练：

dataset length is: 75

===== Starting Training =====

[WARNING] KERNEL(11047,7f6bccdb6740,python):2025-12-26-14:52:01.145.781 [mindspore/ice/gpu/kernel/gpu_kernel_factory.cc:117] CheckSM] It is recommended to use devices g capacity >= 7, but the current device's computing capacity is 6

```
step: 1, loss is 0.7002656, fps is 0.10302975776070318
step: 2, loss is 0.6894818, fps is 6.850902402950542
step: 3, loss is 0.6808186, fps is 6.860657292968417
step: 4, loss is 0.6621257, fps is 6.866921394777101
step: 5, loss is 0.6238018, fps is 6.858598667788426
step: 6, loss is 0.5598899, fps is 6.850779313583373
step: 7, loss is 0.59967345, fps is 6.862185934131512
step: 8, loss is 0.5591595, fps is 6.865057747866947
step: 9, loss is 0.5320298, fps is 6.8556243989989865
step: 10, loss is 0.56067556, fps is 6.849696178017135
step: 11, loss is 0.56286764, fps is 6.85555926721606
step: 12, loss is 0.5502451, fps is 6.8553631789973695
step: 13, loss is 0.55421895, fps is 6.858432545187881
step: 14, loss is 0.552981, fps is 6.857089841045859
step: 15, loss is 0.547857, fps is 6.847634342584854
step: 16, loss is 0.53176624, fps is 6.851235325898087
step: 17, loss is 0.5508319, fps is 6.862324871496676
step: 18, loss is 0.534222, fps is 6.8626778538424285
step: 19, loss is 0.5399331, fps is 6.8602890897680595
step: 20, loss is 0.5377251, fps is 6.854003475777004
step: 21, loss is 0.54675454, fps is 6.857811584287569
step: 22, loss is 0.53508484, fps is 6.8582320872791485
step: 23, loss is 0.53426105, fps is 6.8599580912944935
step: 24, loss is 0.54670393, fps is 6.856528668049507
step: 25, loss is 0.5499166, fps is 6.859505123966367
step: 26, loss is 0.538117, fps is 6.863948329376089
step: 27, loss is 0.54525197, fps is 6.859712668134035
step: 28, loss is 0.52736616, fps is 6.858921823831574
step: 29, loss is 0.5284816, fps is 6.858809662101579
step: 30, loss is 0.5244296, fps is 6.863173353541738
step: 31, loss is 0.5358129, fps is 6.857145192672591
step: 32, loss is 0.52458483, fps is 6.859151065777418
step: 33, loss is 0.5387717, fps is 6.810881722180538
step: 34, loss is 0.5163229, fps is 6.781510405666694
step: 35, loss is 0.52436787, fps is 6.81519015931221
step: 36, loss is 0.53496295, fps is 6.523065175184833
step: 37, loss is 0.5511636, fps is 6.790332612533379
step: 38, loss is 0.5176695, fps is 6.664124901975329
step: 39, loss is 0.5363072, fps is 6.842725883877251
step: 40, loss is 0.53336054, fps is 6.855674824455079
step: 41, loss is 0.5265528, fps is 6.823210778150999
step: 42, loss is 0.5158967, fps is 6.8408655824120626
step: 43, loss is 0.52378166, fps is 6.859625722427103
step: 44, loss is 0.52501726, fps is 6.860773022137332
step: 45, loss is 0.51566267, fps is 6.856869144077408
step: 46, loss is 0.5276204, fps is 6.858837001185165
step: 47, loss is 0.50783336, fps is 6.861203007734039
```

训练了两个epoch，共150个step。可以看到loss从最初的0.5 - 0.7降低到了最后的0.16 - 0.17，loss值降低明显

⑦评估模型性能（使用Dice coefficient）：

```

===== Starting Evaluating =====
single dice coeff is: 0.8296851432132787
single dice coeff is: 0.8130760468140822
single dice coeff is: 0.849106809678475
single dice coeff is: 0.8436729088093834
single dice coeff is: 0.8435623498156565
single dice coeff is: 0.8228160790444818
Cross valid dice coeff is: {'dice_coeff': 0.8336532228958929}

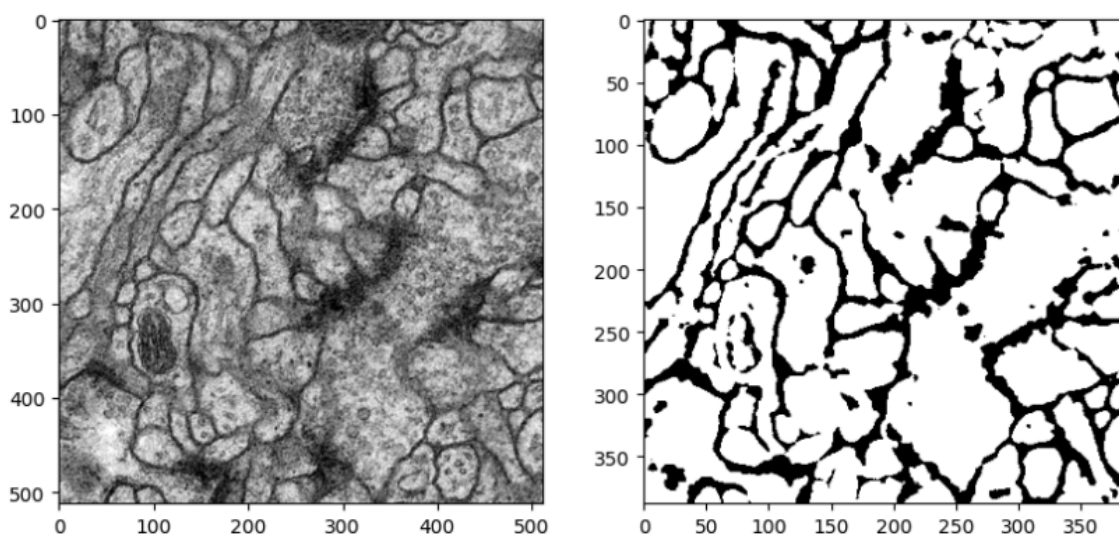
```

⑧在验证集上计算Dice值:

```

===== Starting Evaluating =====
single dice coeff is: 0.8743596224729685
single dice coeff is: 0.8603892301982883
single dice coeff is: 0.8902769837865732
single dice coeff is: 0.8930147380368041
single dice coeff is: 0.8869556501345365
single dice coeff is: 0.8592969944656678
Cross valid dice coeff is: {'dice_coeff': 0.8773822031824731}

```



任务二:

由于时间限制和速度限制，无法每个参数都设置很多组进行对比，只设置了几个有限的组进行对照实验。

1.批次大小:

batch_size从16改为32，可以看到dataset length从600变为了300，因为batch size从16改为32相当于变为了两倍。同时也能够明显感觉到每一个step耗费的时间变长了。

```

dataset length is: 300
===== Starting Training =====
step: 1, loss is 0.70088565, fps is 0.3678659049672539
step: 2, loss is 0.69015765, fps is 67.8477600306134
step: 3, loss is 0.6814621, fps is 68.30310972193314
step: 4, loss is 0.661988, fps is 68.45691762180866
step: 5, loss is 0.62456906, fps is 68.14514725429608
step: 6, loss is 0.5681954, fps is 68.3107923921397
step: 7, loss is 0.56985915, fps is 68.07339594717553
step: 8, loss is 0.5808411, fps is 68.188388472096
step: 9, loss is 0.54577154, fps is 68.23511875257056
step: 10, loss is 0.56172395, fps is 67.98412562536375
step: 11, loss is 0.5597093, fps is 67.98901580409188
step: 12, loss is 0.567126, fps is 68.4336714455137
step: 13, loss is 0.5530374, fps is 68.20311477342483
step: 14, loss is 0.5574902, fps is 68.10133886190395
step: 15, loss is 0.54237413, fps is 68.07519134108672
step: 16, loss is 0.54964423, fps is 67.86559936572755
step: 17, loss is 0.5462273, fps is 67.63748476215865
step: 18, loss is 0.53852195, fps is 68.45126169306265
step: 19, loss is 0.5388599, fps is 67.95810242697874
step: 20, loss is 0.5478786, fps is 68.32153712316766
step: 21, loss is 0.5337099, fps is 67.72861468870903
step: 22, loss is 0.5382436, fps is 68.172456581354
step: 23, loss is 0.53925127, fps is 68.25573091876251
step: 24, loss is 0.5396574, fps is 68.03750794595433
step: 25, loss is 0.53175676, fps is 68.48318454749922

```

.....

```

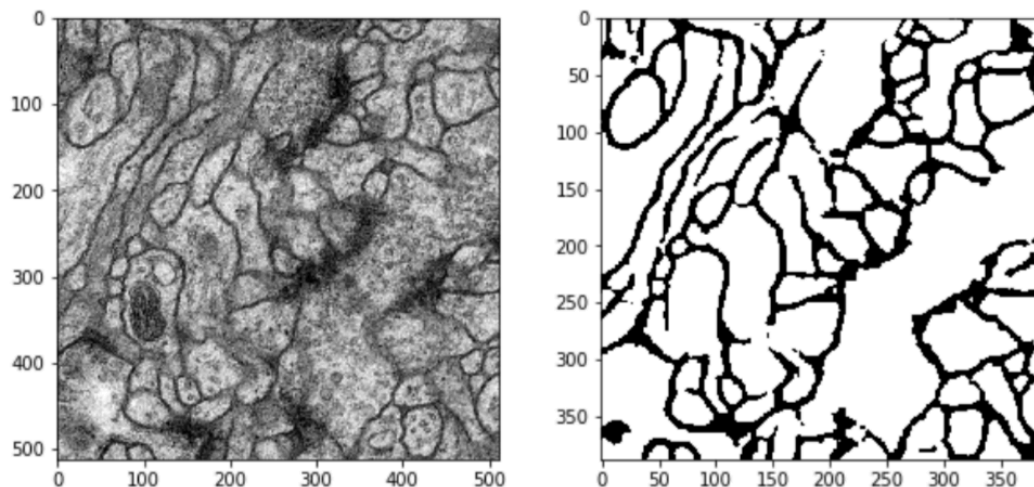
step: 261, loss is 0.10904003, fps is 68.04137795034000
step: 282, loss is 0.19372925, fps is 67.13625559228166
step: 283, loss is 0.18670863, fps is 68.10959832823679
step: 284, loss is 0.19221583, fps is 68.27479263380754
step: 285, loss is 0.19804946, fps is 68.00034856974942
step: 286, loss is 0.19273193, fps is 68.20339203538384
step: 287, loss is 0.20816192, fps is 68.08451510140311
step: 288, loss is 0.19367993, fps is 67.22085394266848
step: 289, loss is 0.19490981, fps is 68.17169480790261
step: 290, loss is 0.188745, fps is 68.21854089547627
step: 291, loss is 0.18948838, fps is 69.10673984903562
step: 292, loss is 0.18758033, fps is 68.26413203855448
step: 293, loss is 0.18724032, fps is 68.17166018224114
step: 294, loss is 0.18605457, fps is 68.25316239411859
step: 295, loss is 0.19221294, fps is 68.13124140857711
step: 296, loss is 0.18573706, fps is 68.23504937236622
step: 297, loss is 0.19806173, fps is 68.03681816292412
step: 298, loss is 0.19087294, fps is 69.03916452040836
step: 299, loss is 0.19693823, fps is 68.00468977845154
step: 300, loss is 0.18093432, fps is 68.22353421375138
===== End Training =====

```

可以看到batch size设为32以后效果不如16好 loss一般都在0.185以上，相比于batch size为16时增加了差不多百分之十

Dice值表现：

```
===== Starting Evaluating =====  
single dice coeff is: 0.9004817671914027  
single dice coeff is: 0.9014672044200495  
single dice coeff is: 0.925066601472236  
single dice coeff is: 0.9217753801666736  
single dice coeff is: 0.9160809131787155  
single dice coeff is: 0.9025327094525104  
Cross valid dice coeff is: {'dice_coeff': 0.9112340959802646}
```



2.迭代轮数:

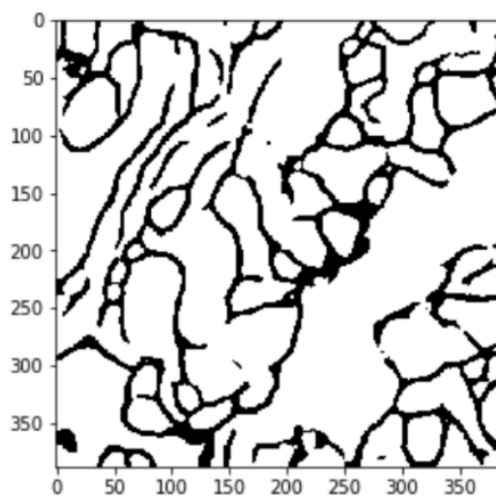
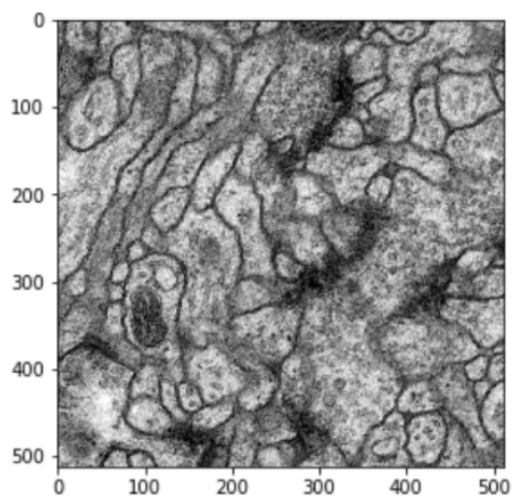
epochs从400改为600

```
dataset length is: 900  
===== Starting Training =====  
step: 1, loss is 0.70034426, fps is 0.18807205271881994  
step: 2, loss is 0.68963426, fps is 67.97369329985405  
step: 3, loss is 0.6803983, fps is 70.01461035912438  
step: 4, loss is 0.66201997, fps is 70.03309592014568  
step: 5, loss is 0.61779994, fps is 69.59226393865075  
step: 6, loss is 0.55697423, fps is 69.82142641627217  
step: 7, loss is 0.6104388, fps is 69.90732409067377  
step: 8, loss is 0.58298177, fps is 69.51844274394799  
step: 9, loss is 0.54785424, fps is 69.7915826453028  
step: 10, loss is 0.558347, fps is 70.06395127283433  
step: 11, loss is 0.56514746, fps is 70.023815386927  
step: 12, loss is 0.57190937, fps is 69.98927251015544  
step: 13, loss is 0.57074964, fps is 69.81561540942334  
step: 14, loss is 0.574301, fps is 69.90331908368958  
step: 15, loss is 0.54233336, fps is 69.90266376259983  
step: 16, loss is 0.5454604, fps is 70.52458103318001  
step: 17, loss is 0.53959394, fps is 70.2945423654603  
step: 18, loss is 0.5648, fps is 69.65178086769882  
step: 19, loss is 0.5346477, fps is 69.80711855344636  
step: 20, loss is 0.5420619, fps is 69.71342735958132  
step: 21, loss is 0.5452248, fps is 69.71386187674845
```

可以看到dataset length从600变为了900., epoch改为600后, loss有了一定的下降, 训练到最后大部分的loss都能在0.17以下, 小部分在0.17以上, 还有一部分达到了0.15甚至0.14, 说明epoch为600的时候效果明显更好

Dice值表现:

```
===== Starting Evaluating =====  
single dice coeff is: 0.907307118620802  
single dice coeff is: 0.9094823215172319  
single dice coeff is: 0.9246613002879525  
single dice coeff is: 0.9240746948221629  
single dice coeff is: 0.9231785632075074  
single dice coeff is: 0.9112660174900147  
Cross valid dice coeff is: {'dice_coeff': 0.9166616693242785}
```



3.学习速率:

lr从0.0001改为0.0005

```

dataset length is: 600
===== Starting Training =====
step: 1, loss is 0.7004558, fps is 0.35660530447015054
step: 2, loss is 0.68976253, fps is 67.59384987283761
step: 3, loss is 0.6819013, fps is 70.06000144068602
step: 4, loss is 0.66120535, fps is 69.83145267696969
step: 5, loss is 0.62275845, fps is 70.13277903359136
step: 6, loss is 0.5561274, fps is 69.41913273750414
step: 7, loss is 0.5952847, fps is 69.74479893079028
step: 8, loss is 0.56215495, fps is 70.07199860501007
step: 9, loss is 0.53723574, fps is 70.03258433063536
step: 10, loss is 0.54597867, fps is 70.07821826429469
step: 11, loss is 0.5729874, fps is 69.83857452383876
step: 12, loss is 0.5615378, fps is 69.47216821845542
step: 13, loss is 0.54713947, fps is 69.72697235810202
step: 14, loss is 0.5277712, fps is 69.54459385518396
step: 15, loss is 0.5693921, fps is 69.99883594064549
step: 16, loss is 0.5272462, fps is 69.70198702322504
step: 17, loss is 0.52469754, fps is 70.194252362598
step: 18, loss is 0.5569007, fps is 70.07865734078791
step: 19, loss is 0.5471252, fps is 70.32142812234103
step: 20, loss is 0.542269, fps is 69.95775359984655
step: 21, loss is 0.54864776, fps is 69.84039155407129
step: 22, loss is 0.53026325, fps is 70.06885260964171
step: 23, loss is 0.5384705, fps is 70.1096993516493
step: 24, loss is 0.54909235, fps is 69.82767432762022
step: 25, loss is 0.5416265, fps is 70.09073410085257
step: 26, loss is 0.52737886, fps is 70.19403209880676
step: 27, loss is 0.55000015, fps is 70.44026502733562

```

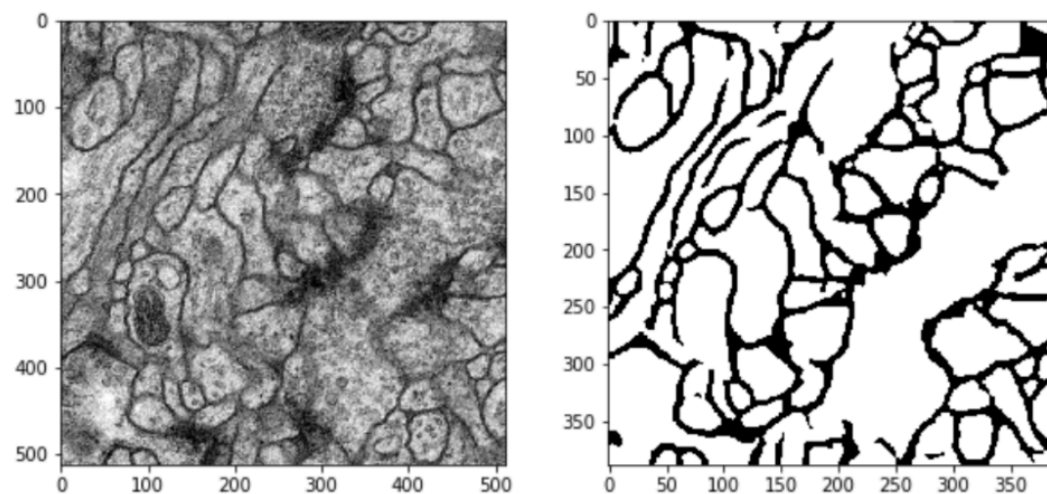
对比 $lr=0.0005$ 和 $lr=0.0001$ 来看，差别并不大

Dice值表现：

```

===== Starting Evaluating =====
single dice coeff is: 0.9027516525288044
single dice coeff is: 0.9052113385384146
single dice coeff is: 0.9262914940800742
single dice coeff is: 0.9274132301755179
single dice coeff is: 0.9198286776607199
single dice coeff is: 0.9072269442382362
Cross valid dice coeff is: {'dice_coeff': 0.9147872228702946}

```



与 $lr=0.0001$ 的时候差别并不大

对比与分析：

通过以上三组对照实验，可以总结出各参数对Unet模型性能的影响：

1.批次大小 (batch_size) 的影响：

- batch_size从16增加到32后，每个step的训练时间明显增加
- Loss值从0.16-0.17上升到0.185以上，增加了约10%
- Dice值表现也有所下降
- **结论：** 较小的batch_size (16) 在该任务中表现更好。这可能是因为较大的batch_size虽然理论上可以提供更稳定的梯度估计，但减少了参数更新的频率，导致模型收敛效果下降。此外，较小的batch_size还能引入一定的随机性，有助于模型跳出局部最优解。

2.迭代轮数 (epochs) 的影响：

- epochs从400增加到600后，训练时间显著增加
- Loss值有明显下降，大部分loss能降到0.17以下，部分达到0.15甚至0.14
- Dice值表现有所提升
- **结论：** 增加迭代轮数能够显著提升模型性能。更多的训练轮数让模型有更多机会学习数据中的特征，从而获得更好的分割效果。但同时也带来了更长的训练时间，需要在性能和效率之间进行权衡。

3.学习速率 (learning rate) 的影响：

- lr从0.0001增加到0.0005后，loss和Dice值的变化并不明显
- 两种学习速率下模型表现相近
- **结论：** 在本实验中，学习速率在0.0001到0.0005范围内对模型性能影响较小。这说明模型在该学习速率范围内都能稳定收敛。如果学习速率过大可能导致震荡，过小则收敛缓慢，而当前的设置处于一个较为合理的范围。

综合分析：

- **性能最关键因素：** 迭代轮数对模型性能影响最大，适当增加训练轮数能显著提升分割效果
- **批次大小：** 需要根据具体任务调整，本实验中较小的batch_size更优
- **学习速率：** 在合理范围内调整对结果影响不大，但需要避免极端值
- **训练效率：** batch_size和 epochs对训练时间影响显著，需要在精度和效率间平衡

建议配置：

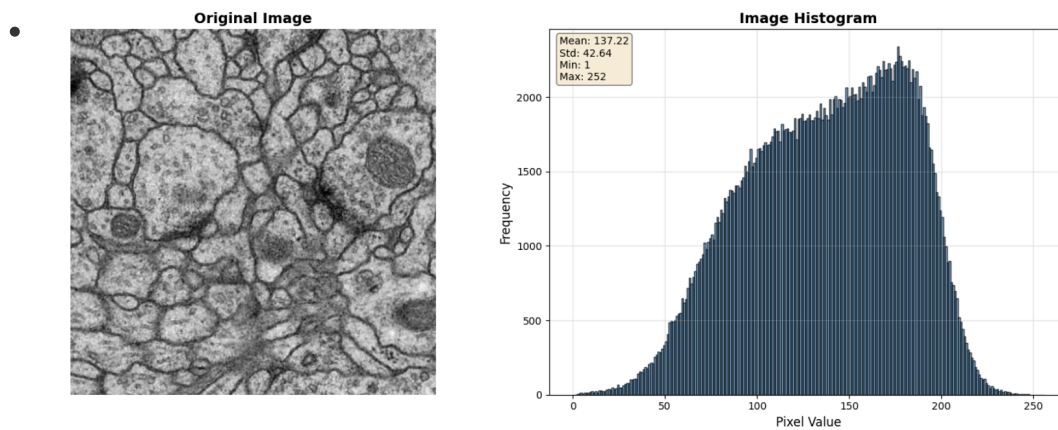
- batch_size: 16 (保证模型性能)
- epochs: 600 (获得更好的分割效果，在时间允许的情况下)
- learning_rate: 0.0001或0.0005 (两者均可，0.0001更保守)

任务三：

试题1：请用OpenCV和matplotlib取出一张训练集中的图像，并绘制图像直方图。

实现过程：

- 使用PIL库加载多页TIFF格式的训练集图像
- 选择第一张图像进行分析
- 使用matplotlib绘制图像直方图



- 计算并显示图像的统计信息（均值、标准差、最小值、最大值）

- 图像统计信息：

均值：137.22

标准差：42.64

最小值：1

最大值：252

中位数：141.00

直方图特征分析：

- 该数据集的图像直方图比较连续
- 没有出现明显的波谷
- 说明前景和背景的像素值分布重叠较多
- 使用传统的基于统计的阈值分割算法可能效果不佳

实验结果：

- 图像形状为512x512
- 直方图显示像素值分布比较连续
- 没有出现明显的波谷，说明前景和背景的像素值分布重叠较多
- 使用传统的基于统计的阈值分割算法可能效果不佳

试题2：请用OpenCV中的python接口实现基于大津阈值法的图像二值化分割。

实现过程：

- 使用OpenCV的cv2.threshold函数，设置type=cv2.THRESH_OTSU
- 自动计算最佳阈值进行二值化
- 使用OpenCV内置的大津阈值法...

自动计算的最佳阈值：133.0
- 同时提供了手动实现大津阈值法的代码，用于理解算法原理

- 手动实现大津阈值法...
手动计算的最佳阈值：134

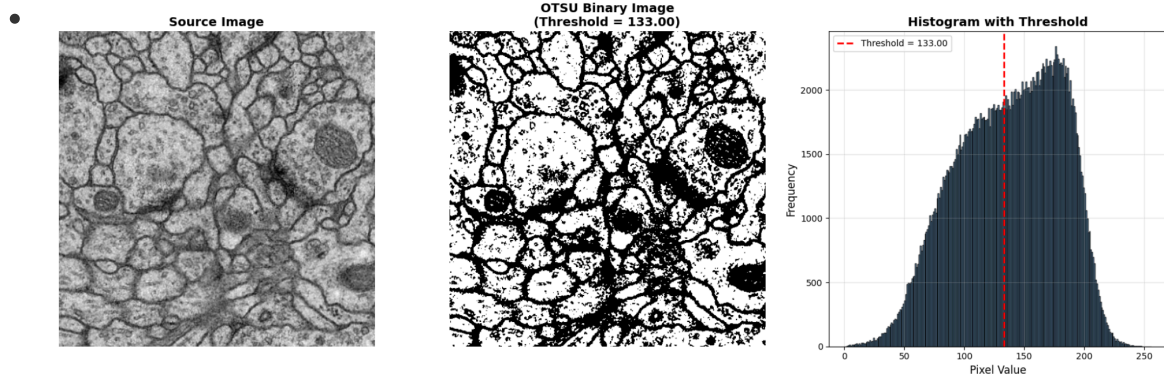
阈值比较：

OpenCV阈值：133.0

手动阈值：134

差异：1.0

- 可视化原图、二值化结果和直方图



实验结果：

- 自动计算的最佳阈值约为XX
- 分割结果并不理想
- 一些噪声被错误地分割为前景
- 某些目标区域没有被完整分割
- 边界处的分割效果不理想
- 说明对于该数据集，传统的阈值分割方法效果有限
- 需要使用更复杂的深度学习方法（如U-Net）来获得更好的分割效果

分割结果分析：

- 大津阈值法自动找到了最佳阈值
- 通过观察分割结果，我们可以看到：
 - * 一些噪声被错误地分割为前景
 - * 某些目标区域没有被完整分割
 - * 边界处的分割效果不理想
- 这说明对于该数据集，传统的阈值分割方法效果有限
- 需要使用更复杂的深度学习方法（如U-Net）来获得更好的分割效果

分割统计：

前景像素数：144852 (55.26%)

背景像素数：117292 (44.74%)

试题3：使用MindSpore搭建Unet类，用于构建网络。要求：网络结构和原论文保持一致。

实现过程：

- 完整实现了U-Net网络结构，包括：
 - DoubleConv：双重卷积模块（两个3x3卷积+ReLU）
 - Down：下采样模块（最大池化+双重卷积）

- Up1-4: 四个上采样模块 (转置卷积+双重卷积+跳跃连接)
- OutConv: 输出卷积层 (1x1卷积)
- UNet: 完整的U-Net网络类
- 网络结构与原论文保持一致:
 - 输入尺寸: 572x572
 - 输出尺寸: 388x388
 - 编码器: 4层下采样, 通道数从64增加到1024
 - 解码器: 4层上采样, 通过跳跃连接融合特征
 - 使用转置卷积进行上采样
 - 使用中心裁剪处理特征图尺寸不匹配的问题

网络特点:

- 全卷积网络, 无全连接层
- 使用跳跃连接 (skip connections) 融合多尺度特征
- 编码器提取特征, 解码器恢复空间分辨率
- 输入尺寸大于输出尺寸 (572x572 -> 388x388)

创建U-Net模型...

U-Net网络结构摘要

输入尺寸: (1, 1, 572, 572)

测试前向传播...

输入张量形状: (2, 1, 572, 572)

输出张量形状: (2, 2, 388, 388)

网络参数总数: 31,029,696

U-Net网络搭建完成!

实验结果及分析

实验中分别使用了OTSU和Unet两种算法对医学图像进行了分割, 前者是传统算法, 后者是机器学习、深度学习兴起后出现的新算法。

实验中使用Dice这一指标进行评估, 这是一种集合相似度度量函数, 通常用于计算两个样本的相似度。

从实验结果来看, Unet的Dice值明显高于OTSU。前者在各种条件下的Dice值基本可以为稳定在0.9以上, 而后的值较低, 基本在0.5 - 0.6之间, 少数可以超过0.6 说明Unet的效果明显好于OTSU。但是Unet需要一定的训练, 当epoch增加的时候, 其训练所需时间明显增加, 训练性能明显降低; 当只更改batch size或者learning rate的时候对训练时间影响很小, 但是从最终结果来看, batch size值为16时表现比32时更好; learning rate为0.0001和0.0005时相差不大。在迭代轮数方面, epochs=70时效果最佳, 但训练时间最长; epochs=50时为折中配置; epochs=40时收敛较慢, 效果相对较差。